

Projeto e Análise de Algoritmos

Corretude de Algoritmos Iterativos

Atílio G. Luiz

Primeiro Semestre de 2026

Invariante de laço

Motivação

- ▶ Ao criar um algoritmo para resolver um determinado problema, esperamos que ele **sempre** dê a resposta correta, qualquer que seja a instância de entrada recebida.
- ▶ Para mostrar que um algoritmo está incorreto, portanto, é simples: basta encontrar uma instância específica para a qual o algoritmo devolve uma resposta errada.
- ▶ Mas como mostrar que o algoritmo está correto?
 - ▶ Em geral a quantidade de instâncias possíveis é infinita.
 - ▶ Assim, é necessário lançar mão de demonstrações matemáticas.

Motivação

- ▶ Alguns algoritmos podem ser bastante simples, tanto que alguma demonstração direta pode ser suficiente.
- ▶ No entanto, estamos mais preocupados com algoritmos mais complexos, que provavelmente envolvem laços (**for** ou **while**).
- ▶ De forma geral, mostraremos que algoritmos iterativos possuem propriedades que continuam verdadeiras antes de (ou após) cada iteração de um determinado laço.
 - ▶ Ao final do algoritmo essas propriedades devem nos ajudar a fornecer uma prova de que o algoritmo agiu corretamente.

Invariante de laço

Definição

Uma **invariante de laço** é uma afirmação que:

- ▶ é satisfeita (verdadeira) em **toda** execução do laço
- ▶ está associada a determinada posição de um laço
- ▶ expressa uma relação entre as variáveis

A posição escolhida é normalmente descrita como

- ▶ imediatamente **antes** ou **depois** da iteração do laço
- ▶ imediatamente **antes** ou **depois** de determinada linha

Objetivos

- ▶ após o término do laço, deve ser uma propriedade útil para se mostrar a correção do algoritmo
- ▶ permite nos concentrar apenas em uma iteração do laço

Um algoritmo

- ▶ O quê o algoritmo abaixo faz?

Algoritmo 3.1: SOMATORIO(A, n)

```
1 soma = 0
2 para  $i = 1$  até  $n$ , incrementando faça
3    $\quad soma = soma + A[i]$ 
4 devolve soma
```

- ▶ O algoritmo recebe um vetor A que armazena n números e promete resolver o problema de calcular a soma dos valores armazenados em A , isto é, promete calcular $\sum_{i=1}^n A[i]$.
- ▶ Como provar isso?

Exemplo de invariante

Algoritmo 3.1: SOMATORIO(A, n)

```
1 soma = 0
2 para  $i = 1$  até  $n$ , incrementando faça
3    $\quad$  soma = soma +  $A[i]$ 
4 devolve soma
```

- ▶ No início da iteração $i = 1$, nenhum elemento do vetor A foi acessado e $soma = 0$
- ▶ No início da iteração $i = 2$, o elemento $A[1]$ do vetor já foi acessado e $soma = A[1]$
- ▶ No início da iteração $i = 3$, os elementos do subvetor $A[1..2]$ já foram acessados e $soma = A[1] + A[2]$
- ▶ ...

Exemplo de invariante

Algoritmo 3.1: SOMATORIO(A, n)

```
1 soma = 0
2 para  $i = 1$  até  $n$ , incrementando faça
3    $soma = soma + A[i]$ 
4 devolve soma
```

Invariante de laço

No início da t -ésima iteração do laço **para** do algoritmo SOMATORIO, temos que $i = t$ e $soma = A[1] + A[2] + \dots + A[i - 1]$

- ▶ Essa afirmação apresenta uma propriedade do algoritmo e gostaríamos de provar que ela é verdadeira no início de qualquer uma das iterações do laço, qualquer que seja o valor da variável i naquele momento.

Exemplo de invariante

Seja $P(t)$ a seguinte afirmação:

$P(t) =$ “no início da t -ésima iteração do algoritmo SOMATORIO, temos que $i = t$ e $soma = A[1] + A[2] + \dots + A[i - 1]$ ”

- ▶ A fim de provar que essa afirmação é verdadeira no início de toda iteração do laço, temos que provar $P(1)$, $P(2)$, $P(3)$, etc.
- ▶ Porém, provar cada uma dessas afirmações separadamente é tedioso e pode levar muito tempo.
- ▶ Para isso, usamos a técnica de **indução matemática**.

Princípio da Indução Matemática

Seja $P(t)$ uma afirmação sobre um número inteiro $t \geq 1$.

A fim de provar que a afirmação $P(t)$ é verdadeira para todo número inteiro $t \geq 1$, basta provar as duas seguintes afirmações:

- (1) $P(1)$ é verdadeira.
- (2) Seja $t > 1$ um inteiro qualquer. Se $P(t)$ é verdadeira, então $P(t+1)$ é verdadeira.

Demonstrando uma invariante de laço

Tipicamente, demonstramos uma invariante com 3 etapas:

1. **Caso Base:** mostre que a propriedade vale antes de qualquer iteração
2. **Passo indutivo:** mostre que, se a propriedade vale no início da j -ésima iteração, então ela também vale no início da $(j + 1)$ -ésima iteração
3. **Término:** conclua que a invariante vale quando o laço termina

Estamos usando o **princípio da indução!**

- ▶ a **base** corresponde à etapa 1
- ▶ o **passo indutivo** corresponde à etapa 2

Demonstração

Invariante de laço

$P(t)$ = “No início da t -ésima iteração do algoritmo SOMATORIO, temos que $i = t$ e $soma = A[1] + A[2] + \dots + A[i - 1]$ ”

Demonstração:

- **Caso Base:** Quando $t = 1$, o teste executou uma vez. Logo antes da iteração começar, temos $soma = 0$ e $i = 1$. Neste momento, $A[1] + A[2] + \dots + A[i - 1] = 0$ pois nenhum elemento de A foi analisado. Logo, $soma = A[1] + A[2] + \dots + A[i - 1]$. Portanto, $P(1)$ é verdadeira e o caso base vale.

Demonstração (cont.)

- ▶ **Passo indutivo:** Considere então que o teste executou t vezes, com $t \geq 1$. Isso significa que a t -ésima iteração vai começar.
- ▶ Suponha que $P(t)$ vale e vamos mostrar que $P(t+1)$ vale. Para isso, argumentamos sobre o que está sendo feito durante a t -ésima iteração.
- ▶ No início da t -ésima iteração, sabemos que $i = t$ e que $soma = A[1] + \dots + A[t-1]$, pois $P(t)$ é verdadeira.
- ▶ Durante a iteração, tudo que é feito é a atualização da variável $soma$, ou seja, é realizada a operação $soma = soma + A[t]$.
- ▶ Assim, o valor da variável $soma$ antes da próxima iteração começar é $soma = A[1] + \dots + A[t-1] + A[t]$.
- ▶ Como na próxima iteração o valor da variável i será $t+1$, acabamos de mostrar que $P(t+1)$ é verdadeira. ■

Invariante de laço

$P(t)$ = “No início da t -ésima iteração do algoritmo SOMATORIO, temos que $i = t$ e $soma = A[1] + A[2] + \dots + A[i - 1]$ ”

- ▶ Portanto, concluímos que $P(t)$ é invariante de laço. Como esse laço executa n vezes, $P(n+1)$ é que nos será útil pois nos dá informação sobre o estado dos dados ao fim da execução do laço.
- ▶ $P(n+1)$ nos diz que $soma = A[1] + \dots + A[n]$, que é justamente o que precisamos para mostrar que o algoritmo está correto.
- ▶ Logo, o algoritmo SOMATORIO soma corretamente todos os elementos de um vetor A com n números.

Exercício

- ▶ Escreva um algoritmo iterativo que encontra o maior elemento de um vetor.
- ▶ Estabeleça uma invariante de laço apropriada para o seu algoritmo.
- ▶ Prove que a invariante de laço é verdadeira para toda iteração do laço.
- ▶ Usando este último fato, conclua que o seu algoritmo é correto

Invariante de laço

- ▶ Correção de INSERTION-SORT

Reverso o INSERTION-SORT

Insertion-Sort(A, n)

```
1  para  $j = 2$  até  $n$  faça
2      chave =  $A[j]$ 
3       $i = j - 1$ 
4      enquanto  $i \geq 1$  e  $A[i] > \textit{chave}$  faça
5           $A[i + 1] = A[i]$ 
6           $i = i - 1$ 
7       $A[i + 1] = \textit{chave}$ 
```

Até agora:

- ▶ já vimos que o algoritmo termina
- ▶ e analisamos sua complexidade de tempo

O que falta fazer?

- ▶ Verificar se ele está *correto*.

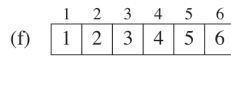
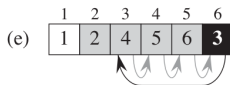
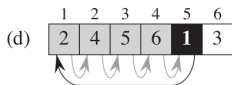
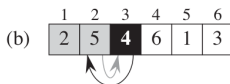
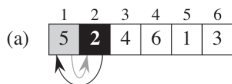
Exemplo de invariante

Insertion-Sort(A, n)

```
1  para  $j = 2$  até  $n$  faça
2       $chave = A[j]$ 
3       $i = j - 1$ 
4      enquanto  $i \geq 1$  e  $A[i] > chave$  faça
5           $A[i + 1] = A[i]$ 
6           $i = i - 1$ 
7       $A[i + 1] = chave$ 
```

- Qual seria um bom invariante para o laço **para**?

Snapshots de um array sob ação do INSERTION-SORT



Em cada j -ésima iteração, o elemento $A[j]$ é guardado em uma variável e comparado com os elementos à sua esquerda (que já estão ordenados) até encontrar a sua posição correta.

Exemplo de invariante

Insertion-Sort(A, n)

```
1  para  $j = 2$  até  $n$  faça
2      chave =  $A[j]$ 
3       $i = j - 1$ 
4      enquanto  $i \geq 1$  e  $A[i] > \textit{chave}$  faça
5           $A[i + 1] = A[i]$ 
6           $i = i - 1$ 
7       $A[i + 1] = \textit{chave}$ 
```

Invariante

Imediatamente antes de cada iteração do laço **para**, o subvetor $A[1 \dots j - 1]$ consiste dos elementos originalmente em $A[1 \dots j - 1]$, ordenados em ordem crescente.

- ▶ **posição da invariante:** antes da iteração do laço **para**
- ▶ **propriedade invariante:** $A[1 \dots j - 1]$ está ordenado

Demonstrando uma invariante: caso base

Considere a primeira iteração do laço para do INSERTION-SORT

- ▶ no início da iteração, temos $j = 2$
- ▶ neste caso, o subvetor $A[1 \dots j - 1]$ contém apenas um elemento
- ▶ $A[1]$ é o primeiro elemento do vetor original e está ordenado
- ▶ então, a invariante vale antes da primeira iteração

Demonstrando uma invariante: passo indutivo

Suponha que a invariante vale no início de **alguma iteração j**

- ▶ nessa iteração, temos **$chave = A[j]$**
- ▶ informalmente, o corpo do laço **enquanto** desloca os valores $A[j-1], A[j-2], A[j-3]$, e assim por diante, uma posição para a direita, até encontrar a posição correta para **$chave$** , que é a posição $i+1$.
- ▶ Os valores em $A[1 \dots i]$ são todos menores ou iguais que **$chave$** e os valores em $A[i+2 \dots j]$ são todos maiores que a **$chave$** e ambos subarrays estão ordenados
- ▶ Após o laço **enquanto**, $A[i+1]$ recebe o valor de **$chave$**
- ▶ Assim, o subarray $A[1 \dots j]$ consiste dos elementos originalmente em $A[1 \dots j]$, mas em ordem crescente.
- ▶ Incrementando j para a próxima iteração do laço **para** preserva a invariante de laço

Demonstrando uma invariante: término

Finalmente, examinamos o que acontece quando o laço termina

- ▶ o laço **para** termina quando $j > n$
- ▶ como cada iteração incrementa j em 1, então $j = n + 1$ na última iteração
- ▶ Substituindo $n + 1$ por j no enunciado da invariante de laço, temos que: o subarray $A[1 \dots n]$ consiste dos elementos originalmente em $A[1 \dots j]$, em ordem crescente.
- ▶ Assim, o array inteiro está ordenado e o algoritmo é correto.